

Đơn giản hóa quan hệ dữ liệu

He Lin

2005

Nguồn gốc: Simplifying data relations

IOI China candidate team papers / 2005 / He Lin / He Lin.doc

Abstract

Quan hệ giữa dữ liệu quan trọng không kém bản thân dữ liệu. Các quan hệ thường gặp nhất là quan hệ tuyến tính, quan hệ cây và quan hệ đồ thị. Bản chất của thi tin học là khai thác đầy đủ dữ liệu, nhưng đôi khi khai thác quá đầy đủ lại làm bài toán phức tạp hơn. Bài viết này bàn về cách đơn giản hóa quan hệ dữ liệu một cách thích hợp để khai thác dữ liệu hợp lý.

Từ khóa: cây, đồ thị, dãy, quan hệ dữ liệu.

1 Dẫn nhập

Ta thường đối mặt với lượng lớn dữ liệu, nhưng dữ liệu không hẳn là vô trật tự. Những thành phố nối với nhau bằng đường tạo thành quan hệ đồ thị; quan hệ cấp trên - cấp dưới trong cơ quan là quan hệ cây; xếp hạng điểm số ở trường là quan hệ tuyến tính.

Đồ thị, cây và quan hệ tuyến tính là ba loại quan hệ thường gặp nhất trong đời sống cũng như trong thi tin học. Dù thi tin học nhấn mạnh việc khai thác và sử dụng đầy đủ thông tin đầu vào, đôi khi quan hệ quá phức tạp lại khiến ta mất phương hướng. Tư tưởng chính ở đây là đơn giản hóa cấu trúc dữ liệu: có thể biến đồ thị thành cây, biến cây thành cấu trúc tuyến tính. Trong quá trình này chắc chắn mất đi một phần thông tin, nhưng các ví dụ dưới đây cho thấy đó là một hướng tư duy khả thi.

2 Đơn giản hóa quan hệ đồ thị

2.1 Bài toán đi thuyền

Trường trung học Yali có n học sinh đi công viên chèo thuyền. Mỗi thuyền tối đa chở hai người. Nếu hai học sinh cùng họ hoặc cùng tên, họ có thể ngồi chung thuyền. Nhà trường muốn mọi học sinh đều lên thuyền, nhưng chi phí thuê thuyền cao, nên cần thuê ít thuyền nhất.

Một cách tự nhiên là xem mỗi học sinh là một đỉnh. Nếu hai học sinh cùng họ, nối họ bằng cạnh đỏ; nếu cùng tên, nối bằng cạnh xanh. Khi đó bài toán trở thành tìm phủ cạnh nhỏ nhất, tương đương với tìm ghép cực đại. Nhưng đồ thị không nhất thiết là hai phía, nên ghép cực đại cần thuật toán blossom, rất phức tạp.

Giả sử trước hết đồ thị liên thông. Ta dùng một cây nhị phân để biểu diễn đồ thị: con trái của mỗi nút, nếu tồn tại, có cùng họ với nút đó; con phải, nếu tồn tại, có cùng tên.

Cách dựng cây:

1. Giả sử cây hiện có k đỉnh. Nếu $k = n$, kết thúc.
2. Vì đồ thị liên thông, trong các đỉnh chưa vào cây có một đỉnh P nối với một đỉnh T đã ở trong cây.
3. Nếu P và T cùng họ, đi liên tục theo con trái từ T đến khi gặp đỉnh X không có con trái. Vì X cùng họ với T , X cũng cùng họ với P ; đặt P làm con trái của X .
4. Nếu P và T cùng tên, làm tương tự theo con phải.

Như vậy luôn có thể tăng kích thước cây thêm 1 cho đến khi mọi đỉnh đều nằm trong cây.

Sau khi có cây, xét một lá P , cha của nó là F . Nếu P là con duy nhất của F , cho P và F đi chung thuyền, rồi xóa cả hai khỏi cây. Nếu F có hai con, gọi con còn lại là Q . Không mất tính tổng quát, giả sử P là con trái, Q là con phải.

Nếu F là gốc, còn ba học sinh chưa lên thuyền; ít nhất cần hai thuyền. Cho F và P đi chung, còn Q đi một thuyền.

Nếu F không phải gốc, xét cha F' của nó. Trường hợp F là con trái của F' : cho F và Q đi chung, xóa F, Q khỏi cây. Vì F' cùng họ với F , và F cùng họ với P , nên F' cũng cùng họ với P ; đặt P làm con trái của F' , cây vẫn giữ được tính chất. Trường hợp F là con phải làm tương tự với quan hệ cùng tên.

Lặp lại quá trình trên sẽ liên tục giảm kích thước cây. Với cây có n đỉnh, số thuyền cuối cùng là

$$\left\lceil \frac{n}{2} \right\rceil,$$

rõ ràng là tối ưu. Nếu đồ thị không liên thông, xử lý từng thành phần liên thông rồi cộng kết quả.

Độ phức tạp của thuật toán là $O(n)$, tốt hơn nhiều so với thuật toán ghép, và độ phức tạp cài đặt cũng đơn giản hơn hẳn. Điều đáng chú ý là ta đã bỏ qua nhiều cạnh trong đồ thị, nhưng quan hệ được giữ lại đủ để giải bài toán tối ưu.

2.2 Nhận diện đồ thị xương rồng

Một đồ thị có hướng được gọi là **xương rồng** nếu:

- nó liên thông mạnh;
- mỗi cạnh thuộc đúng một chu trình.

Cho đồ thị có n đỉnh và m cạnh, với $1 \leq n, m \leq 10^5$, cần kiểm tra nó có phải đồ thị xương rồng hay không.

Một thuật toán dễ nghĩ là liên tục tìm một chu trình, kiểm tra giữa các đỉnh không kề trong chu trình có cạnh phụ hay không; nếu không thì co chu trình thành một đỉnh rồi tiếp tục. Nhưng chỉ riêng việc co đỉnh và cập nhật quan hệ giữa chu trình, đỉnh và cạnh đã rất phức tạp.

Cách khác là chạy DFS và xét cây DFS. Nếu đồ thị là xương rồng, cây DFS của nó có các tính chất sau.

Tính chất 1. Cây DFS của đồ thị xương rồng không có cạnh ngang. Thật vậy, nếu có cạnh ngang từ một nhánh sang nhánh khác, do đồ thị liên thông mạnh, sẽ tồn tại đường quay lại. Dù đường này giao hay không giao với đường tổ tiên tương ứng, một số cạnh sẽ thuộc hai chu trình, mâu thuẫn với định nghĩa xương rồng.

Tính chất 2. Đặt $\text{DFS}(v)$ là thứ tự thăm của v trong DFS. Đặt $\text{Low}(v)$ là giá trị DFS nhỏ nhất có thể đi tới bằng một cạnh đi ra trực tiếp từ v hoặc từ các hậu duệ của v . Với mọi cạnh cây (v, u) , trong đó u là

con của v , phải có

$$\text{Low}(u) \leq \text{DFS}(v).$$

Nếu không, cạnh (v, u) là cầu, điều không thể xảy ra trong đồ thị liên thông mạnh.

Tính chất 3. Với mỗi đỉnh v , gọi $a(v)$ là số con u của v thỏa $\text{Low}(u) < \text{DFS}(v)$, và $b(v)$ là số cạnh ngược từ chính v . Khi đó phải có

$$a(v) + b(v) < 2.$$

Nếu tổng này từ 2 trở lên, sẽ có cạnh thuộc hai chu trình.

Ba tính chất trên cho ta thuật toán DFS tuyến tính. Nếu cây DFS vi phạm bất kỳ tính chất nào, đồ thị không phải xương rồng. Ngược lại, nếu thỏa tất cả, có thể chứng minh bằng phân tích cạnh cây và cạnh ngược rằng đồ thị là xương rồng. Độ phức tạp thời gian là $O(n + m)$.

Điểm bản chất là ta mô tả lại đồ thị dưới dạng cây DFS. Với cây, quan hệ tổ tiên - hậu duệ rõ ràng hơn, và các cạnh ngược, cạnh ngang có thể được xử lý tách biệt. Hình thức đơn giản hơn làm quan hệ dữ liệu nổi lên rõ ràng.

3 Đơn giản hóa quan hệ cây

3.1 Thống kê trên cây

Cho một cây có n đỉnh, đánh số $1, 2, \dots, n$. Với đỉnh v , định nghĩa $t(v)$ là số hậu duệ của v có số hiệu nhỏ hơn v . Cần in

$$t(1), t(2), \dots, t(n).$$

Thuật toán trực tiếp $O(n^2)$ quá chậm. Ta có thể giải bằng một cách khác: biến cây thành hai dãy.

Thực hiện DFS và ghi lại các đỉnh theo thứ tự thăm, thu được **dãy DFS**. Sau đó đảo thứ tự các con của mỗi đỉnh rồi DFS lại, thu được **dãy DFS ngược**. Với một đỉnh v , phần sau v trong dãy DFS bao phủ một vùng, phần sau v trong dãy DFS ngược bao phủ một vùng khác; giao của hai vùng này chính là tập hậu duệ của v , còn phần bị tính lệch liên quan đến chính v và các tổ tiên trực tiếp của nó.

Định nghĩa $f(v, S)$ là số đỉnh trong tập hoặc vùng S có số hiệu nhỏ hơn v . Khi đó theo nguyên lý bao hàm - loại trừ:

$$\begin{aligned} t(v) &= f(v, \text{phần sau } v \text{ trong dãy DFS}) \\ &\quad + f(v, \text{phần sau } v \text{ trong dãy DFS ngược}) \\ &\quad + f(v, \text{các tổ tiên trực tiếp của } v) - (v - 1). \end{aligned}$$

Số tổ tiên trực tiếp nhỏ hơn v có thể tính bằng một lần DFS kết hợp cây đoạn. Với hai dãy DFS, số phần tử phía sau nhỏ hơn phần tử hiện tại có thể tính bằng cây đoạn hoặc Fenwick. Vì vậy toàn bộ bài toán được giải trong

$$O(n \log n).$$

Điểm tinh tế là cây được ánh xạ thành hai dãy. Quan hệ hậu duệ trong cây vốn có tính phân tầng; sau biến đổi, nó trở thành quan hệ trước - sau trong dãy, và nhờ đó có thể dùng cấu trúc dữ liệu tuyến tính quen thuộc.

3.2 Tổ tiên chung gần nhất

Cho một cây n đỉnh và nhiều truy vấn dạng: tổ tiên chung gần nhất của p và q là gì?

Ta biểu diễn cây bằng một dãy Euler. Nếu cây chỉ có một đỉnh, dãy là (*Root*). Nếu gốc có các cây con tương ứng với các dãy

$$L_1, L_2, \dots, L_t,$$

thì dãy của cây là

$$(Root, L_1, Root, L_2, Root, \dots, L_t, Root).$$

Ghi kèm độ sâu của mỗi lần xuất hiện. Khi cần tìm LCA của p, q , chọn một lần xuất hiện bất kỳ của p và một lần xuất hiện bất kỳ của q trong dãy. Trong đoạn giữa hai vị trí đó, đỉnh có độ sâu nhỏ nhất chính là tổ tiên chung gần nhất.

Chứng minh cũng theo đệ quy. Giả sử kết luận đúng với mọi cây con $L_1, \dots, L_t$. Nếu p, q nằm trong cùng một cây con L_i , kết luận theo giả thiết quy nạp. Nếu $p \in L_i, q \in L_j$ với $i \neq j$, LCA của chúng là *Root*. Trong cách dựng dãy, giữa hai dãy con bất kỳ đều có một lần xuất hiện của *Root*, và gốc có độ sâu nhỏ nhất, nên kết luận đúng.

Do đó bài toán LCA được đưa về bài toán RMQ trên dãy độ sâu. Có thể dùng cây đoạn để trả lời mỗi truy vấn trong $O(\log n)$, hoặc dùng RMQ tĩnh để trả lời trong $O(1)$.

4 Kết luận

Chủ đề của bài viết là đơn giản hóa quan hệ dữ liệu. Hai dạng đơn giản hóa quan trọng là từ đồ thị sang cây, và từ cây sang dãy. Những ví dụ trên cho thấy đây không phải là việc bỏ thông tin một cách tùy tiện, mà là thay đổi hình thức biểu diễn để làm rõ mâu thuẫn cốt lõi của bài toán.

Nói nghiêm ngặt, “đơn giản hóa” chỉ là một biến đổi hình thức: dùng cấu trúc đơn giản và nguyên thủy hơn để biểu diễn quan hệ vốn được thể hiện bằng cấu trúc phức tạp. Sự đơn giản này thường làm quan hệ dữ liệu rõ hơn, tạo điều kiện để nhìn thấy bản chất bài toán. Đặc biệt, dùng cây DFS và dãy DFS để xử lý các bài toán đồ thị, cây là một tư tưởng rất thực dụng và hiệu quả.

Ngoài DFS còn có nhiều ứng dụng khác của việc đơn giản hóa quan hệ dữ liệu. Chẳng hạn trong lý thuyết đồ thị, kiểm tra đồ thị đầy cung bằng **thứ tự loại bỏ hoàn hảo** cũng là một ví dụ tốt.

Tài liệu tham khảo

- Đề thi tỉnh Hồ Nam.
- Bài viết của Xiao Zhou, đội tuyển tập huấn quốc gia tin học năm 2000.
- *Introduction to Algorithms*.
- Bài toán gốc của Lei Tao, Trường trung học Yali, Trường Sa, Hồ Nam.